

Sweep Line — Complete Study Guide

Transcribed from handwritten/annotated notes

Sweep Line — Complete Guide

1. What is it?

Sweep line is a technique where you imagine a vertical (or horizontal) line **sweeping across a set of events sorted by position** (usually time, x-coordinate, or a numeric range boundary), processing events in order and maintaining some running state (a count, a running sum, an active set) as the line passes each event.

It converts a 2D or interval-overlap problem into a **1D ordered event-processing problem**.

2. Why is it used?

- Turns $O(n^2)$ pairwise interval/segment comparisons into $O(n \log n)$ sorted processing
- Naturally handles “how many things are active at time X” without re-scanning
- Works well when the *only* thing that matters is order of events, not their exact geometry
- Combines cleanly with a heap, ordered map (TreeMap), or difference array to track “currently active” state

3. Where is it used?

- Interval scheduling / meeting rooms (min rooms needed, can attend all meetings)
- Merge intervals, insert interval
- Skyline problem (building silhouette)
- Maximum overlapping intervals / max concurrent events
- Calendar booking (detect double-booking, k-booking)
- Car pooling / trip overlap capacity problems
- Computational geometry: closest pair of points, segment intersection, rectangle area/union
- Range update problems solvable via difference arrays

4. How to identify a Sweep Line problem (pattern triggers)

Signal in the problem statement	What it suggests
"intervals", "meetings", "events with start/end"	Classic sweep line candidate
"maximum number of overlapping...", "min resources/rooms needed"	Event counting sweep
"at any point in time", "concurrently"	Sweep with a running counter
"skyline", "silhouette", "union of rectangles"	Sweep + ordered structure (heap/TreeMap)
"book a meeting without conflict", "k-th booking allowed"	Sweep with a count threshold
Multiple [start, end] pairs given	Strong signal — sort by start (and sometimes end)
"on a number line", "range queries applied to array"	Difference array / sweep over deltas

Paraphrase test: If the problem can be rephrased as “process a bunch of start/end markers in order and track how many are open at once” — it's sweep line.

5. Can you tell from constraints alone?

Yes:

- Large n (up to 10^5 - 10^6) with intervals/points given → sorting-based $O(n \log n)$ sweep is expected, since brute-force pairwise ($O(n^2)$) is too slow.
- If the problem only asks for a count/aggregate at each moment (not exact geometric shapes) → simple counter sweep suffices, no heap needed.
- If it asks for the actual shape/skyline or “which interval is active” → you need an ordered structure (heap or TreeMap) alongside the sweep, not just a counter.
- If updates apply to *ranges* of an array and you only need the final array → difference array + prefix sum (a sweep variant) beats doing per-index updates.

6. Intuition / Rationale

A sweeping line works because overlap/interval problems have a hidden 1D structure: you don't actually care about the shapes, you care about *the sequence of moments where the state changes* — a start “opens” something, an end “closes” it. Between two consecutive events, nothing changes, so you never need to examine the interior of an interval — only its boundaries.

This is the same monotonicity idea as two pointers: once you sort events, you process each boundary exactly once, and the running state (count, sum, or active set) is updated incrementally rather than recomputed from scratch.

7. Time & Space Complexity (generic)

- **Time:** $O(n \log n)$ — dominated by sorting events (or the heap operations if used)

- **Space:** $O(n)$ for the event list / heap / active-interval structure

8. Types of Sweep Line (with Java templates)

Type A — Counting Sweep (max concurrent events)

Used for: meeting rooms II, max overlapping intervals, car pooling capacity check.

```
public int minMeetingRooms(int[][] intervals) {
    int n = intervals.length;
    int[] starts = new int[n];
    int[] ends = new int[n];
    for (int i = 0; i < n; i++) {
        starts[i] = intervals[i][0];
        ends[i] = intervals[i][1];
    }
    Arrays.sort(starts);
    Arrays.sort(ends);

    int rooms = 0, maxRooms = 0;
    int s = 0, e = 0;
    while (s < n) {
        if (starts[s] < ends[e]) { // a meeting starts before another ends
            rooms++;
            s++;
        } else { // a meeting ends, free a room
            rooms--;
            e++;
        }
        maxRooms = Math.max(maxRooms, rooms);
    }
    return maxRooms;
}
```

Type B — Event List with +1/-1 Deltas

Used for: same problems as Type A, but generalizes better when you need to process by exact timestamp order (ties handled explicitly — ends before starts at the same time, to free a room before opening a new one).

```

public int minMeetingRoomsDelta(int[][] intervals) {
    List<int[]> events = new ArrayList<>(); // {time, delta}
    for (int[] iv : intervals) {
        events.add(new int[]{iv[0], 1}); // start: +1 active
        events.add(new int[]{iv[1], -1}); // end: -1 active
    }
    // Sort by time; if tie, process ends (-1) before starts (+1)
    events.sort((a, b) -> a[0] != b[0] ? a[0] - b[0] : a[1] - b[1]);

    int active = 0, maxActive = 0;
    for (int[] ev : events) {
        active += ev[1];
        maxActive = Math.max(maxActive, active);
    }
    return maxActive;
}

```

Type C — Sweep with a Heap (track which interval ends soonest)

Used for: skyline problem, whether a new meeting fits in an existing room, k-booking.

```

public boolean canAttendAll(int[][] intervals) {
    Arrays.sort(intervals, (a, b) -> a[0] - b[0]); // sort by start
    PriorityQueue<Integer> endTimes = new PriorityQueue<>(); // min-heap of end times

    for (int[] iv : intervals) {
        if (!endTimes.isEmpty() && endTimes.peek() <= iv[0]) {
            endTimes.poll(); // free up the room that already ended
        }
        endTimes.offer(iv[1]);
    }
    return endTimes.size() <= 1; // or compare against room capacity
}

```

Type D — Difference Array (range updates on a fixed-size array/line)

Used for: "add value to all indices in [l, r]" applied many times, then read final array.

```

public int[] applyRangeUpdates(int n, int[][] updates) {
    int[] diff = new int[n + 1];
    for (int[] u : updates) {
        int l = u[0], r = u[1], val = u[2];
        diff[l] += val;
        diff[r + 1] -= val; // cancels the effect after r
    }
    int[] result = new int[n];
    int running = 0;
    for (int i = 0; i < n; i++) {
        running += diff[i];
        result[i] = running;
    }
    return result;
}

```

Type E — Coordinate Compression + Sweep (large/sparse coordinate ranges)

Used for: skyline, rectangle union area, when coordinates span huge ranges but there are few distinct values.

```
public int maxOverlapCompressed(int[][] intervals) {
    TreeMap<Integer, Integer> deltaMap = new TreeMap<>(); // sorted by coordinate
    for (int[] iv : intervals) {
        deltaMap.merge(iv[0], 1, Integer::sum);
        deltaMap.merge(iv[1], -1, Integer::sum);
    }
    int active = 0, maxActive = 0;
    for (int delta : deltaMap.values()) { // TreeMap iterates in sorted key order
        active += delta;
        maxActive = Math.max(maxActive, active);
    }
    return maxActive;
}
```

9. Comparison of the patterns

Type	Structure used	Typical trigger words	Complexity
A. Counting (two sorted arrays)	two pointers on sorted starts/ends	"min rooms", "max overlap"	$O(n \log n)$
B. Event list with deltas	sorted list of (time, ± 1)	same as A, but needs tie-breaking control	$O(n \log n)$
C. Heap-based sweep	min-heap of end times	"skyline", "k-booking", "can attend"	$O(n \log n)$
D. Difference array	plain array + prefix sum	"apply range update", fixed small coordinate range	$O(n + q)$
E. Coordinate compression + TreeMap	sorted map of deltas	huge/sparse coordinate range, still need overlap count	$O(n \log n)$

Key distinguishing question: Do I just need a running count ($\rightarrow$ Type A/B), do I need to know which specific interval is "active" or ending soonest ($\rightarrow$ Type C heap), am I applying repeated range updates to array indices ($\rightarrow$ Type D), or are coordinates too large/sparse for a plain array ($\rightarrow$ Type E)?

10. Quick decision checklist (use this in an interview)

1. Do I only need a max/min concurrent count? $\rightarrow$ **Type A or B**
2. Do I need to know *which* interval is ending soonest, or simulate limited resources? $\rightarrow$ **Type C (heap)**
3. Am I applying many range-add updates to indices of a bounded array? $\rightarrow$ **Type D (difference array)**

4. Are the coordinates huge or sparse (not a small bounded range)? → **Type E (coordinate compression)**
5. Does the problem also require me to reconstruct the actual overlapping shape (e.g. skyline outline), not just a count? → **Type C**, output changes on every heap-top update, not just count changes

If ties at the same coordinate need special handling (e.g. does a meeting ending at 10 conflict with one starting at 10?), always decide this explicitly in your sort comparator — it's the most common sweep-line bug.

11. Comparison with Two Pointers

	Two Pointers	Sweep Line
Core idea	Two indices moving toward/along a single sorted array	One (conceptual) pointer moving through sorted events derived from multiple intervals/points
State tracked	Usually a running sum/comparison between two values	Running count, active set, or heap of "currently open" items
Typical output	Pair/triplet, boolean, or a single optimal value	Count of overlaps, min resources, or reconstructed shape
Data shape	Flat array	Intervals, points, or ranges (2 numbers per item)
When they overlap	Type A sweep is two pointers, just over two separate sorted arrays (starts, ends) instead of one array from both ends	—

Sweep line is often thought of as “two pointers generalized to events with a start and an end,” which is why Type A above looks almost identical to a two-pointer template.